Idiomagic:
Comparing a RAG and an LLM-Only Dialogue System

Elisa Beihofer
University of Gothenburg
gusbeihel@student.gu.se

1 The Idea

For my final project, I created an LLM-augmented dialogue system in SpeechState that can explain English idioms and their meanings in context. The idea is that the user can ask about an idiom, say in which context they heard it and/or ask for examples as well as more details (e.g.register and difficulty) about the idiom.

Since LLMs can struggle with pragmatic use of language, I decided to use a RAG-system to make the dialogue system more reliable. In order to see if that is necessary, I compared the RAG-system with a dialogue system that only uses an LLM, but no database. The question I want to answer is which of the two systems answers more reliably.

My project therefore includes the building of an idiom explainer application, as well as the comparison between a RAG and an LLM-only dialogue system.

2 Building the Dialogue Systems

I will first describe the architecture of my *Idiomagic* idiom explainer systems.

2.1 RAG-System

2.1.1 Data

For my RAG system, I had to find a suitable dataset that contains English idioms as well as explanations and examples. After comparing several datasets, I chose the IdiomX dataset from HuggingFace (2). This dataset includes many English idioms sourced from online dictionaries and additional data such as meaning, examples, explanations, and more. According to the creator, it is meant to help AI systems understand idiomatic language and supports tasks like idiom detection and understanding. The full dataset features idioms in English, Arabic and French. Overall, it is well-suited to my task. However, parts of the dataset include LLM-generated enrichment, which the author acknowledges as a limitation.

I pre-processed the dataset in *data.ipynb*. I first selected the relevant English data fields since the original dataset provided too much additional data for my purposes. I grouped multiple instances of the same idiom together and added the examples from each instance to the single remaining idiom entry. The final dataset includes the idiom, its meaning and explanation, the register, difficulty, and all examples. For each example, the meaning and explanation, as well as the context type, tags, if it is LLM-generated, and the validation status are recorded.

Here is an example (truncated):


```
{
  "idiom": "in someone's face",
  "meaning": "Confronting someone boldly,
    aggressively, or offensively without
    regard for social norms or personal
    boundaries.",
  "idiom_explanation": "The phrase 'in someone
    's face' is idiomatic because it expresses
     figurative confrontation or aggressive
    behavior that goes beyond the literal
    spatial meaning of being physically in
    front of someone.",
  "register": "informal",
  "difficulty": "medium",
  "examples": [
    {
      "sentence": "During the debate, she put
    her arguments right in my face, leaving no
     room for doubt.",
      "meaning": "She confronted me directly
    and assertively with her arguments in a
    challenging way.",
      "explanation": "Figurative confrontation
     expressing assertiveness and challenge in
     discourse.",
      "context_type": "formal",
      "tags": "idiomatic",
      "is_generated": true,
      "validation_status": "valid"
    },
    {
      "sentence": "He tried not to be in her
    face, but his constant presence felt
    overwhelming.",
```

```
      "meaning": "Negation of aggressive
    confrontation, but with a nuance of
    discomfort due to proximity.",
      "explanation": "The phrase is negated to
     challenge models recognizing idiomatic
    confrontation, blending literal presence
    with implied aggression.",
      "context_type": "adversarial",
      "tags": "idiomatic",
      "is_generated": true,
      "validation_status": "valid"
    }
  ]
}
```


In total, there were 15 075 grouped records after pre-processing, which I saved in *idioms_grouped.json*. I am aware that there are some unusual idioms in the dataset. However, the size of the dataset did not allow me to manually check the quality of each idiom and its additional data, especially the examples. This, along with the LLM-enrichment of the original dataset, is a possible limitation to the quality of the dataset.

After finalizing the dataset, I used Qdrant as a local vector database running in a Docker container via *docker-compose.yml*. I created a new Qdrant collection called *idioms* to store the 384-dimensional vectors. I then added *addIdioms* as a new command to the Lab 1 *main.ts* script to import my JSON data into Qdrant. I used the *qwen3-embedding* model via Ollama (accessed on university server through SSH tunnel) to generate a 384-dimensional vector for each idiom record. Each idiom is stored as a Qdrant point containing its embedding and structured payload (meaning, explanation, register, difficulty, examples).

2.1.2 Implementation

The next step was to implement a RAG system in my SpeechState/XState dialogue system. My Lab 1 code served as the starting point for my RAG idiom explainer. I had already implemented a *queryRAG* actor that embeds the last user utterance, queries Qdrant, and returns the retrieved record to the dialogue manager. The retrieved information is then passed to an LLM, in this case *llama3.2:latest*, and included in the system prompt. After generation, the generated text is passed to a dialogue state and spoken by the system. For my final project, I kept this structure and modified/expanded it.

I first implemented intent recognition. Using Azure NLU, I trained it to recognize various intents like asking for a new idiom explanation, providing context, asking for examples and for more details, recognizing no, and understanding that the user does not have a context. Instead of manually training Azure NLU for entity extraction, I chose to use the LLM to extract the idiom and possibly the context from the user utterances, returned as JSON. After extracting the idiom, Qdrant is queried with it and a matching entry is selected. Qdrant is only queried again if the user switches to a new idiom.

To allow for all the different dialogue options I had planned, I added several states to the dialogue system. They handle asking for missing idioms, follow-up questions, missing context, and more. The main deicision state is *Greeting*. Every intent can be detected here, therefore this state routes to the different dialogue paths. After each task is performed, the system asks "Anything more?" and routes back to *Greeting*. Other important states are *ExtractRequest* + *extractRequest* actor, *Retrieve*, *GiveContext*, and *GetCompletion*.

A special case to handle is when the user asks for an example or does not have a context of their own. If the retrieved idiom record includes at least one example, my dialogue system will provide that. If there are several examples, the system will select them in order in *GiveContext* and keep track of which examples have already been used. I used the keywords *literal* and *literally* in the explanations to filter out possible literal examples. This is very simple, however, and might not filter out all literal examples or filter out idiomatic examples too.

The LLM prompt had to be updated in order for the system to correctly use the retrieved Qdrant data. For this, I used conditional instructions to tell the LLM which parts of the data to use for generating its reponse. I manually set a *dialogueIntent* depending on which dialogue state is entered, and used this to construct the conditional LLM prompt. This lets the LLM know which task it should perform (explain selected data example, explain user context, provide more details or give a general explanation). The LLM is prompted to only use the retrieved data / the user context for its explanations. It should not make up anything else.

Prompting the LLM posed some problems. Initially, when prompted to give and explain an example selected from the dataset, the LLM did not follow this prompt and often generated its own example and explained this instead. I decided to make the dialogue system simply say the example word by word and then added the LLM-generated explanation afterwards to guarantee that the example stays as it is. This still did not fix the problem of the LLM generating different examples and ex-

planations. Prompt engineering did not show any remarkable improvements. In the end, the fix was to not give the LLM the entire dialogue history, but only the data it was supposed to explain.

I also improved the robustness of the system by handling errors, no input, clarification, and resetting the dialogue variables between sessions / when a new idiom is introduced. Then I tried to optimize idiom retrieval. Now the top five results are retrieved and the system tries to find a match for the LLM-extracted idiom by allowing exact or partial (contained) matches with the retrieved results. This does, however, not include different spellings or variants of idioms in which the extracted idiom is not included.

Lastly, I updated the user interface to not only display the button that starts the conversation, but also the current idiom, the user context, and the selected example.

The system can now explain an idiom with user context, provide non-duplicate examples (if available in the database), reject idioms that are not in the database, switch to a different idiom, clarify when no intent is matched with sufficient accuracy, and provide additional details about an idiom (if in the retrieved data) in a multi-turn dialogue. Different dialogue variants are supported, e.g. providing context first and the idiom only in the second turn.

2.2 LLM-Only System

In order to create the LLM-only system, I adapted my RAG system in *starter-llm* and removed all RAG components. This included Qdrant, the *queryRAG* actor, the *Retrieve* state, variables used to keep track of the retrieved data, as well as changes to some dialogue states and the omitting of others. The two systems were kept as similar as possible. Especially the LLM prompts were only changed as much as necessary.

The LLM does not receive any retrieved data in this dialogue system, and instead has to generate explanations, examples and more details about the idiom itself. It receives the whole dialogue history to prevent duplicate examples.

3 Limitations - Dialogue Systems

The two primary challenges for the functionality of the dialogue systems are Azure intent detection and using an LLM to extract the idiom and context from the user utterance. If the intent detection fails or detects a wrong intent, it might break the entire dialogue flow. This tends to happen when it does not identify certain idioms as a request to explain a new idiom. If the LLM fails to extract the idiom or the context correctly, this leads to an incorrect dialogue (from the user's point of view). Sometimes, the LLM also hallucinates a context even though none was provided, or edits the provided context (against instructions). This also leads to incorrect dialogue.

Furthermore, the system does not ask clarification questions about the user's context and therefore sometimes hypothesizes too much about about possible meanings in the context.

An issue specifically for the RAG system is that the extracted idiom has to match the retrieved idiom exactly or be contained in it for the system to accept a retrieved match. This is very exclusive and can lead to an idiom being rejected even if there is an entry in the database - but in slightly different wording.

4 Comparing RAG and LLM-Only

I compared the two systems using ten different dialogues. I had each conversation three times with each system. I reloaded the application and tried to keep my wording as similar as possible between each attempt and between both systems. The ten different dialogues aimed to test different things such as a common idiom with or without context, an uncommon idiom, several examples, etc. (see *system_comparison.pdf* for all tasks and dialogue transcriptions).

Afterwards I scored each test for each task on a 0 - 1- 2 scale for meaning accuracy, contextual accuracy (when applicable), example correctness (when applicable), explanation clarity, and factual reliability. A score of 0 meant not correct/appropriate, a score of 1 partially correct/appropriate, and a score of 2 fully correct/appropriate. The highest possible score for each system was 420.

4.1 Results & Discussion

The RAG system achieved a score of 361 out of 420 (85.95%) and the LLM-only system achieved a score of 324 (77.14%). Variation between individual tasks occurred, but overall, the RAG system scored better when explaining idioms, their contexts, and providing examples. This suggests that the RAG system is the more reliable choice for explaining idioms.

However, while the RAG system provides more

reliable and correct explanations, it is more limited than the LLM-only system: it only accepts idioms that are in its finite database, and even these have to be very close matches. This makes it more restrictive than the LLM-only system.

While the LLM-only system provides greater flexibility and accepts all idioms, it is also more prone to hallucinations and inconsistencies. It sometimes produces weird examples like: *"Under the weather" means feeling unwell or ill, often with a cold or flu. Your friend is saying she's not feeling great. The speaker, likely someone else, is saying your friend is ill, but might not be the one describing the situation.* Here it is inconsistent about who the speaker is. Some of the generated examples also do not feel quite right or natural, for example: *"I really liked your presentation, you kept the nail on the head and explained the complex points in a way that was easy to understand."* This should have been: *"hit the nail on the head"*.

While the LLM-only system is more prone to generating slightly wrong utterances and explanations, this does not mean that the RAG system is always better. It also uses an LLM that might generate wrong or slightly off things. In general, it is more stable though, since it relies on the retrieved data, which gives it less room for error.

Overall, I think both systems performed reasonably well. While they still rely on LLM-generated explanations (with our without the support of a more reliable dataset) and therefore do not always truly capture human meaning and intentions, they deliver good explanations for most contexts and examples.

5 Limitations

The comparison of the two systems has only been done with 30 examples so far, of which only a third are novel conversations. For a more robust analysis, the comparison should be done with more example conversations.

The scoring of the results was only done by me, with no reviewer or a second annotator. This makes the judgments prone to subjectivity.

Furthermore, the systems were only tested on idioms that were in the RAG system's dataset. Therefore, this test only looked at the quality of the generated answers, not at the overall performance of the systems in a real-world setting. Idiom rejections and misunderstandings were not reflected in these tests. The practical reliabilty of the two dialogue systems needs to be tested separately.

6 Future Work

In future work, I would want to run tests to compare the two systems on a bigger scale with more testers and human judgments. I would also want to ensure the quality of the data used for the RAG system. This could be done via manual judgments or filtering the dataset using existing, reliable idiom lists.

Additionally, the dialogue systems can be expanded to provide a better user experience. I could add a feature to allow the comparison of different idioms from the same conversation or similar idioms and a feature that allows the user to learn about a random idiom if they do not have a specific one in mind. I would also implement *bargeIn* to allow the user to interrupt the system, and I would want to use a different, more capable LLM to see if that would improve the results. For the RAG system specifically, I would like to make the idiom matching more inclusive and allow non-verbatim matches.

References

[1] Sharara, A. A. (2026). *IdiomX: A Multilingual Benchmark for Idiom Understanding, Retrieval, and Interpretation*. arXiv preprint arXiv:2606.02584. https://arxiv.org/abs/2606.02584

[2] Sharara, A. A. (2026). *IdiomX Dataset*. Zenodo. https://doi.org/10.5281/zenodo.19137833